🎤 Voice Coding Assistant — Complete Guide

> ***An AI-powered VS Code extension that lets you code entirely using voice commands.*** *Designed for users with hand disabilities — no keyboard or mouse required. Built with special support for* ***Indian English accents****,* ***Hindi confirmations****, and* ***hands-free continuous operation****.*

📑 Table of Contents

1. Project Overview
2. Project Structure
3. Architecture
4. Module Responsibilities
5. Installation Guide
6. Running the Extension
7. Voice Commands Reference
8. Continuous Listening & Wake Word
9. Voice Feedback (TTS)
10. Indian Accent Support
11. Safety & Confirmation System
12. Keyboard Shortcuts
13. Settings & Configuration
14. Supported Programming Languages
15. Run Commands by Language
16. How Each Module Works
17. Testing Voice Commands
18. Troubleshooting

🌟 Project Overview

The **Voice Coding Assistant** is a VS Code extension that translates spoken English into code editor actions. It is designed from the ground up for **accessibility** — enabling developers with hand disabilities, RSI, or mobility challenges to write, navigate, debug, and run code without ever touching a keyboard or mouse.

Key Features

- 🎙️ **Voice-to-Code** — Speak commands like "*create a python file*" or "*add a for loop*"
- 🧠 **AI-Powered Code Generation** — Uses OpenRouter (GPT-4, Claude, DeepSeek, etc.) for intelligent code generation
- 🔊 **Voice Feedback (TTS)** — The assistant speaks back confirmations so you know what happened
- 👂 **Continuous Hands-Free Mode** — Say "*Hey Coder*" to wake, give commands, and it listens continuously
- IN **Indian Accent Optimized** — Tuned regex patterns and Whisper configuration for Indian English speakers
- 🛡️ **Safety Prompts** — Destructive actions (delete file, replace code) require spoken confirmation
- 🌍 **22+ Programming Languages** — Python, Java, JavaScript, TypeScript, C++, Go, Rust, and more
- 💬 **Hindi Confirmation Support** — Say "*haan*" (हाँ) for yes, "*nahi*" (नहीं) for no

---

📁 Project Structure

```
voice-coding-assistant/
├── .env                        ← Your OpenRouter API key
```

```
├── .gitignore
├── .vscode/
│   ├── launch.json             ← F5 debug configuration
│   └── tasks.json              ← TypeScript watch task
├── src/
│   ├── extension.ts            ← Main entry point (orchestrator)
│   ├── voiceRecorder.ts        ← Microphone recording via SoX
│   ├── speechToText.ts         ← Audio → Text via Whisper
│   ├── commandParser.ts        ← Text → Structured command (the brain)
│   ├── commandDictionary.ts    ← 120+ command reference & help system
│   ├── aiEngine.ts             ← AI code generation via OpenRouter
│   ├── syntaxDetector.ts       ← Language detection & syntax rules
│   ├── vscodeController.ts     ← Execute actions in VS Code (the hands)
│   ├── wakeWordDetector.ts     ← Wake word & deactivation phrase detection
│   ├── voiceFeedback.ts        ← Text-to-speech spoken confirmations
│   └── voiceModeManager.ts     ← Passive/Active listening mode management
├── out/                        ← Compiled JavaScript (auto-generated)
├── package.json                ← Extension manifest & dependencies
├── tsconfig.json               ← TypeScript configuration
├── requirements.txt            ← Python dependencies
└── node_modules/               ← Installed packages
```

🏗️ Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                     Voice Coding Assistant v2.0                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ┌──────────────┐                                                │
│  │ 🎤 SoX       │ ← Records 16kHz mono WAV from microphone       │
│  │ voiceRecorder│                                                │
│  └──────┬───────┘                                                │
│         │                                                        │
│         ▼                                                        │
│  ┌──────────────┐                                                │
│  │ 📝 Whisper   │ ← Converts speech → text (local, private)      │
│  │ speechToText │    Handles Indian, American, British accents   │
│  └──────┬───────┘                                                │
│         │                                                        │
│         ├──────────────────────────────┐                         │
│         │                              │                         │
│         ▼                              ▼                         │
│  ┌──────────────┐                ┌──────────────┐                │
│  │ 👂 Wake Word │ (PASSIVE mode) │ 🧠 Command   │ (ACTIVE mode)  │
│  │ Detector     │ Listens for    │ Parser       │ Processes full │
│  │              │ "Hey Coder"    │              │ voice commands │
│  └──────┬───────┘                └──────┬───────┘                │
│         │                               │                         │
│         │ Wake word detected!           ├──────────────┐          │
│         │ → Switch to ACTIVE            │              │          │
```

```
│          └──────────────┘          ┌──────┬──────┐  ┌──────┬──────┐   │
│                                    │ Direct      │  │ AI-Needed   │   │
│                                    │ Commands    │  │ Commands    │   │
│  ┌──────────┐                      │ save, nav,  │  │ insert,     │   │
│  │ 🗣️ Voice    │ ← Speaks          │ close, run  │  │ debug, fix  │   │
│  │ Feedback TTS │  confirmations    └──────┬──────┘  └──────┬──────┘   │
│  └──────────┘                             │               │          │
│                                           │               │          │
│  ┌──────────┐                             │         ┌──────┬──────┐   │
│  │ 🔍 Syntax    │ ← Detects language        │         │ 🤖 AI Engine │   │
│  │ Detector     │  for correct code         │         │ OpenRouter   │   │
│  └──────────┘                             │         └──────┬──────┘   │
│                                           │               │          │
│                                           │               │          │
│                                    ┌──────┴───────────────┘          │
│                                    ▼                                 │
│                            ┌──────────┐                              │
│                            │ 🖥️ VS Code  │ ← Creates files,          │
│                            │ Controller │   navigates, inserts       │
│                            │          │   code, runs programs       │
│                            └──────┬──────┘                          │
│                                   │                                  │
│                                   ▼                                  │
│                            ✅ Action Executed                        │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Two Listening Modes

Mode	What Happens	CPU Usage	Purpose
PASSIVE	Short recordings (2s), only checks for wake word	Low	Always-on background listening
ACTIVE	Longer recordings (7s), full command processing	Normal	Processing actual voice commands

```
PASSIVE ──"Hey Coder"──→ ACTIVE ──commands──→ ACTIVE
                                                  │
                                 30s timeout or   │
                                 "stop coding" ──→ PASSIVE
```

📋 Module Responsibilities

#	Module	File	Purpose
1	**Extension**	`extension.ts`	Main orchestrator — registers commands, ties all modules together
2	**Voice Recorder**	`voiceRecorder.ts`	Records audio from microphone using SoX (16kHz mono WAV)

3	**Speech-to-Text**	`speechToText.ts`	Converts WAV audio to text using Whisper (Python subprocess)
4	**Command Parser**	`commandParser.ts`	Interprets text into structured commands using priority-ordered regex
5	**Command Dictionary**	`commandDictionary.ts`	120+ command reference for help system and validation
6	**AI Engine**	`aiEngine.ts`	Sends prompts to OpenRouter API for code generation/debugging
7	**Syntax Detector**	`syntaxDetector.ts`	Detects language from file extension, provides syntax rules to AI
8	**VS Code Controller**	`vscodeController.ts`	Executes VS Code actions: file ops, navigation, code insertion
9	**Wake Word Detector**	`wakeWordDetector.ts`	Detects "Hey Coder" and deactivation phrases with fuzzy matching
10	**Voice Feedback**	`voiceFeedback.ts`	Text-to-speech confirmations using OS-native engines
11	**Voice Mode Manager**	`voiceModeManager.ts`	Manages PASSIVE ↔ ACTIVE mode transitions and listening loops

---

🔧 Installation Guide

Prerequisites

Software	Version	Purpose
Node.js	v20+	Extension runtime
Python	3.8+	Whisper speech recognition
SoX	14.4.2+	Microphone recording
FFmpeg	any	Audio processing for Whisper
Whisper	latest	Speech-to-text model

STEP 1: Install Node.js

1. Download from: https://nodejs.org/ (LTS version)
2. Install with default settings
3. Verify:

```
node --version    # Expected: v20.x.x or higher
npm --version     # Expected: 10.x.x or higher
```

STEP 2: Install Python

1. Download from: https://www.python.org/downloads/

2. ⚠️ **Check "Add Python to PATH" during installation**
3. Verify:

```
python --version    # Expected: Python 3.8 or higher
```

STEP 3: Install SoX (Sound eXchange)

1. Download from: https://sourceforge.net/projects/sox/files/sox/14.4.2/
2. Download `sox-14.4.2-win32.exe` for Windows
3. Install it
4. **Add SoX to your PATH:**
   - Open System Settings → Environment Variables
   - Edit `Path` → Add `C:\Program Files (x86)\sox-14-4-2\`
5. Restart VS Code
6. Verify:

```
sox --version    # Expected: sox: SoX v14.4.2
```

STEP 4: Install FFmpeg

1. Download from: https://www.gyan.dev/ffmpeg/builds/ (Windows builds)
2. Extract the archive
3. **Add FFmpeg `bin` folder to PATH** (e.g., `C:\ffmpeg\bin`)
4. Verify:

```
ffmpeg -version    # Expected: ffmpeg version X.X.X
```

STEP 5: Install Whisper

```
pip install openai-whisper
```

Verify:

```
python -c "import whisper; print(whisper.__version__)"
# Expected: A version number (e.g., 20231117)
```

STEP 6: Get OpenRouter API Key

1. Go to: https://openrouter.ai
2. Sign up / Log in
3. Go to: https://openrouter.ai/keys
4. Click **"Create Key"**
5. Copy the key

STEP 7: Configure the Extension

Create/edit the `.env` file in the project root:

```
OPENROUTER_API_KEY=sk-or-v1-your-key-here
```

STEP 8: Install Dependencies & Compile

```
cd d:\assistant\voice-coding-assistant
npm install
npm run compile
```

Expected result: `out/` folder is created with compiled JavaScript. Zero errors.

🚀 Running the Extension

Launch in Debug Mode

1. Open the project in VS Code: `d:\assistant\voice-coding-assistant`
2. Press **F5** (or Run → Start Debugging)
3. A new VS Code window opens — the **Extension Development Host**
4. Your extension is now active in this new window

> 💡 ***Tip:*** *The Extension Development Host is a separate VS Code instance. After making code changes, press* ***F5*** *again to restart it.*

Verify Activation

1. Press **Ctrl+Shift+P** to open the Command Palette
2. Type **"Voice Coding"** — you should see 7 commands:
   - `Voice Coding: Hello World (Test)`
   - `Voice Coding: Start Listening (Single Command)`
   - `Voice Coding: Stop Listening`
   - `Voice Coding: Type Command (Manual)`
   - `Voice Coding: Start Continuous Listening (Hands-Free)`
   - `Voice Coding: Activate Now (Skip Wake Word)`
   - `Voice Coding: Toggle Voice Feedback (TTS)`
3. Run **"Voice Coding: Hello World"** to verify the extension is working.

🗣️ Voice Commands Reference

📄 File Operations (15 commands)

Voice Command	What Happens
"create a python file"	Creates `main.py` and opens it
"create a java file named Calculator"	Creates `Calculator.java`
"create a javascript file"	Creates `main.js`
"create a c plus plus file"	Creates `main.cpp`

"create file named utils"	Creates `utils.py` (default: Python)
"open file main.py"	Opens `main.py` from workspace
"open utils"	Searches and opens matching file
"save file" / "save this" / "save"	Saves the active file
"save all files" / "save all"	Saves all open files
"close file" / "close this" / "close tab"	Closes the active tab
"run file" / "run this" / "run"	Executes the file in terminal
"delete file temp"	Deletes `temp` (asks confirmation first)
"reopen file" / "reopen last file"	Reopens the last closed file
"undo close"	Reopens the last closed tab

> 🇮🇳 ***Indian accent support:*** *You can say "please create a python file", "kindly save file", "can you open file main" — polite prefixes are recognized.*

🧭 Code Navigation (15 commands)

Voice Command	What Happens
"go to line 10"	Moves cursor to line 10
"go to line twenty five"	Spoken numbers work too
"navigate to line 42"	Alternative phrasing
"jump to line 7"	Another alternative
"line 5"	Shorthand — just say the line
"go to top" / "go to beginning"	Moves cursor to line 1
"go to bottom" / "go to end"	Moves cursor to last line
"next function"	Jumps to next function definition
"previous function" / "last function"	Jumps to previous function
"find variable count"	Searches and navigates to `count`
"find function add"	Finds the `add` function
"search for total"	Finds `total` in the file
"locate class Calculator"	Finds the `Calculator` class

✏️ Code Generation — AI-Powered (20 commands)

Voice Command	What AI Generates

"create function add numbers"	`def add_numbers(a, b): ...`
"insert function"	Empty function skeleton
"define variable count"	`count = 0`
"create class Animal"	Class with docstring
"add constructor with name and age"	`__init__(self, name, age)`
"create loop" / *"add for loop"*	For loop structure
"add while loop"	While loop structure
"add if condition"	If-else block
"add if condition to check if number is positive"	Specific conditional
"call function add"	Function call: `add()`
"add error handling" / *"add try catch"*	Try/except block
"add try except"	Python-style error handling
"add getter and setter methods"	Accessor methods
"create function to sort an array"	Complete sorting function
"write a binary search function"	Binary search implementation
"create a linked list class"	Full linked list with methods

> 💡 ***Tip:*** *The AI detects the language of the open file and generates code in the correct syntax. Open a* `.java` *file and say "create function add" — you'll get Java code, not Python.*

🐛 Debugging & Error Fixing (10 commands)

Voice Command	What Happens
"debug error in line 15"	AI analyzes code at line 15
"debug line 10"	AI checks code around line 10
"debug line twenty"	Spoken numbers work
"explain this error" / *"explain error"*	AI explains what's wrong
"what's the error"	AI explains the error
"fix this error" / *"fix the bug"*	AI fixes and replaces the code
"fix the issue" / *"fix the problem"*	Same as above
"analyze error" / *"check for errors"*	General error analysis

🏷️ Variable & Symbol Operations (10 commands)

Voice Command	What Happens
"rename variable x to y"	Renames all `x` → `y` in file
"rename function old to new"	Renames function throughout
"rename class Dog to Animal"	Renames class throughout
"rename count to total"	No type keyword needed
"import os"	Inserts `import os` at top
"import module requests"	Inserts `import requests`
"import numpy"	Inserts `import numpy`
"import json"	Inserts `import json`

▶️ Program Execution (10 commands)

Voice Command	What Happens
"run file" / *"run this"* / *"run"*	Runs the current file
"run code" / *"execute file"*	Same as above
"stop program" / *"stop execution"*	Kills running terminal
"stop running" / *"kill process"*	Same as above
"open terminal" / *"new terminal"*	Opens integrated terminal
"show terminal" / *"launch terminal"*	Same as above

🖥️ Editor Control (10 commands)

Voice Command	What Happens
"undo" / *"undo that"*	Undoes last action
"redo" / *"redo that"*	Redoes last undone action
"select all" / *"select everything"*	Selects all text
"copy" / *"copy this"*	Copies selection
"paste" / *"paste here"*	Pastes from clipboard
"switch file" / *"switch tab"*	Switches to another tab
"next tab" / *"next file"*	Goes to next open file
"delete line" / *"delete line 5"*	Deletes a line
"delete lines 5 to 10"	Deletes a range of lines
"cut line" / *"duplicate line"*	Cut or duplicate current line

📁 Project Management (10 commands)

Voice Command	What Happens
"*create new project*"	Opens folder picker
"*create folder utils*"	Creates `utils/` directory
"*create folder components*"	Creates `components/` directory
"*make folder models*"	Creates `models/` directory

🤖 AI Free-Form Generation (Fallback)

Any command that doesn't match the above patterns is automatically sent to the AI for code generation:

Example	What AI Generates
"*write a hello world program*"	Complete hello world
"*generate a REST API endpoint*"	API boilerplate
"*write unit tests for add function*"	Test code
"*create a login form validation*"	Validation code
"*generate database CRUD operations*"	DB operations
"*write a web scraper*"	Web scraping code

> 📊 ***Total supported commands:*** *120+ (including aliases and variations)*

👂 Continuous Listening & Wake Word

Starting Continuous Mode

1. Press **Ctrl+Shift+C** or run "*Voice Coding: Start Continuous Listening*"
2. The extension enters **PASSIVE** mode — listening in the background
3. Say **"Hey Coder"** to activate
4. Give your commands (ACTIVE mode)
5. After 30 seconds of silence, it returns to PASSIVE mode
6. Say **"stop coding"** or **"goodbye"** to deactivate manually

Wake Words (What Activates the Assistant)

Phrase	Notes
"*Hey Coder*"	Primary wake word
"*Hey Coda*"	Whisper mis-transcription (accepted)
"*Hey Koder*"	Indian accent: k/c variation
"*Hey Codar*"	Indian accent: trailing 'ar'
"*Hei Coder*"	Indian accent: 'hey' → 'hei'

"Hay Coder"	Indian accent: 'hey' → 'hay'
"Start Coding"	Natural phrasing
"Wake Up"	Simple wake
"Activate Assistant"	Formal phrasing

Deactivation Phrases (What Puts It to Sleep)

Phrase	Notes
"Stop Coding"	Primary deactivation
"Goodbye" / *"Goodbye Coder"*	Natural phrasing
"Go to Sleep"	Friendly phrasing
"Stop Listening"	Direct
"Deactivate" / *"Sleep"*	Short commands
"Bye Bye" / *"Bye Coder"*	Indian English common

Combined Wake + Command

You can say the wake word AND a command together:

> *"Hey Coder, create a python file"*

The assistant will activate and immediately execute the command.

🔊 Voice Feedback (TTS)

The assistant speaks back confirmations using your operating system's built-in text-to-speech engine.

Examples of What It Says

Action	Spoken Feedback
File created	*"File created successfully"*
File saved	*"File saved"*
Voice activated	*"Voice assistant activated. Listening for commands."*
Voice deactivated	*"Going to sleep. Say Hey Coder to wake me."*
Delete confirmation	*"Are you sure you want to delete this file? Say yes to confirm."*

Configure Voice Feedback

Setting	Default	Range
`voiceFeedbackEnabled`	`true`	on/off

`voiceFeedbackRate`	`5`	0 (slow) → 10 (fast)
`voiceFeedbackVolume`	`80`	0 (silent) → 100 (max)

Toggle via Command Palette: **"Voice Coding: Toggle Voice Feedback (TTS)"**

Platform-Specific TTS Engines

OS	Engine	Notes
Windows	PowerShell `System.Speech`	Built-in, no install needed
macOS	`say` command	Built-in, no install needed
Linux	`espeak`	May need: `sudo apt install espeak`

IN Indian Accent Support

This extension is specifically optimized for Indian English speakers. Here's what was done:

Whisper Model Recommendation

Accent	Recommended Model	Notes
American English	`base`	Works well out of the box
British English	`base`	Works well out of the box
Indian English	`small` or `medium`	**Significantly better accuracy**
Australian English	`base`	Works well

> ⚠️ ***Important:*** *For Indian accents, change your Whisper model in VS Code Settings:* `Voice Coding Assistant > Whisper Model` → `small` *or* `medium`

Indian Pronunciation Handling

The command parser handles common Indian English pronunciation patterns:

Indian Pronunciation	Standard	What It Matches
"wariable" / *"wariabul"*	"variable"	`define wariable count` → `INSERT_CODE` ✅
"hey codar"	"hey coder"	Wake word activated ✅
"hei coder"	"hey coder"	Wake word activated ✅
"hay coder"	"hey coder"	Wake word activated ✅
"twinty"	"twenty"	`go to line twinty` → line 20 ✅
"fife"	"five"	`go to line fife` → line 5 ✅
"thurty"	"thirty"	`go to line thurty` → line 30 ✅

Polite Prefix Support

Indian English commonly uses polite prefixes. All are recognized:

Voice Command	Result
"*please create a python file*"	Creates python file ✅
"*kindly save file*"	Saves file ✅
"*can you create a function*"	Creates function ✅
"*could you open file main*"	Opens main file ✅
"*please run file*"	Runs file ✅

Hindi Confirmation Support

For safety prompts (like file deletion), you can respond in Hindi:

Hindi	English	Meaning
"*haan*" (हाँ)	"yes"	Confirms action
"*ha*" (हा)	"yes"	Confirms action
"*theek hai*" (ठीक है)	"okay"	Confirms action
"*nahi*" (नहीं)	"no"	Cancels action
"*mat karo*" (मत करो)	"don't do it"	Cancels action
"*ruko*" (रुको)	"stop"	Cancels action

🛡️ Safety & Confirmation System

Destructive Actions Require Confirmation

These commands ask "*Are you sure?*" before executing:

- **Delete file** → "*Say yes to confirm deletion*"
- **Replace all code** (error fix) → "*This will replace your code. Confirm?*"

Confirmation Flow

```
User: "delete file main.py"
  ↓
Assistant (TTS): "Are you sure you want to delete main.py? Say yes to confirm."
  ↓
User: "yes" / "haan" / "confirm" / "go ahead"
  ↓
Assistant: Deletes file ✓
  ↓
Assistant (TTS): "File deleted successfully."
```

```
User: "delete file main.py"
  ↓
Assistant (TTS): "Are you sure? Say yes to confirm."
  ↓
User: "no" / "nahi" / "cancel" / "abort"
  ↓
Assistant: Action cancelled.
```

Fallback System (When Speech Fails)

```
Recording attempt
    │
    ├── ✅ Success → Show result, ask for confirmation
    │       ├── "Execute" → Run the command
    │       ├── "Retry Recording" → Record again
    │       ├── "Type Instead" → Manual text input
    │       └── "Cancel" → Stop
    │
    └── ❌ Failure (no speech detected)
            │
            ├── Attempt 1 of 3 → Retry / Type / Cancel
            ├── Attempt 2 of 3 → Retry / Type / Cancel
            └── Attempt 3 of 3 → Type Command / Cancel
                                    │
                                    └── Opens text input box
                                        User types: "create python file"
                                        → Parsed and executed normally
```

> 💡 ***You're never stuck:*** *Even if the microphone fails, you can always use* ***Ctrl+Shift+T*** *to type commands manually.*

⌨️ Keyboard Shortcuts

Shortcut	Action
Ctrl+Shift+V	Start voice recording (single command)
Ctrl+Shift+X	Stop recording
Ctrl+Shift+T	Type command manually (text input)
Ctrl+Shift+C	Start continuous listening (hands-free mode)
Ctrl+Shift+A	Activate now (skip wake word)

⚙️ Settings & Configuration

Open VS Code Settings (**Ctrl+,**) and search for **"Voice Coding"**:

Core Settings

Setting	Default	Description
`openRouterModel`	`deepseek/deepseek-chat`	AI model for code generation
`recordingDurationMs`	`7000` (7 seconds)	Recording duration for active commands
`whisperModel`	`base`	Whisper model size (see below)

Voice Feedback Settings

Setting	Default	Description
`voiceFeedbackEnabled`	`true`	Enable/disable spoken confirmations
`voiceFeedbackRate`	`5`	Speaking rate (0 = slow → 10 = fast)
`voiceFeedbackVolume`	`80`	Volume (0 = silent → 100 = max)

AI Model Options

Model	Cost	Quality	Speed	Best For
`deepseek/deepseek-chat`	Very Low	Excellent	Fast	**Recommended for coding**
`openai/gpt-3.5-turbo`	Low	Good	Fast	General use
`openai/gpt-4o`	High	Best	Medium	Complex code
`anthropic/claude-3.5-sonnet`	Medium	Excellent	Medium	Detailed explanations
`google/gemini-2.5-flash`	Low	Good	Fast	Quick tasks
`meta-llama/llama-3.1-8b-instruct`	Free	Good	Fast	Budget option

Whisper Model Sizes

Model	Parameters	Accuracy	Speed	Accent Support
`tiny`	39M	Basic	Fastest	English only
`base`	74M	Good	Fast	Good for standard accents
`small`	244M	Better	Medium	**Good for Indian accents**
`medium`	769M	Very Good	Slow	**Best for Indian accents**
`large`	1550M	Best	Slowest	All accents

> 🇮🇳 ***Recommendation for Indian users:*** *Set Whisper model to* `small` *or* `medium` *for best results.*

📝 Supported Programming Languages

The assistant can create files in and generate code for **22+ languages**:

Language	Extension	Voice Example	Run Command

Python	`.py`	"*create a python file*"	`python file.py`
JavaScript	`.js`	"*create a javascript file*"	`node file.js`
TypeScript	`.ts`	"*create a typescript file*"	`npx ts-node file.ts`
Java	`.java`	"*create a java file*"	`javac file.java && java Class`
C#	`.cs`	"*create a c sharp file*"	`dotnet run`
C++	`.cpp`	"*create a c plus plus file*"	`g++ file.cpp -o a.out && a.out`
C	`.c`	"*create a c file*"	`gcc file.c -o a.out && a.out`
Go	`.go`	"*create a go file*"	`go run file.go`
Rust	`.rs`	"*create a rust file*"	`cargo run`
Ruby	`.rb`	"*create a ruby file*"	`ruby file.rb`
PHP	`.php`	"*create a php file*"	`php file.php`
Swift	`.swift`	"*create a swift file*"	`swift file.swift`
Kotlin	`.kt`	"*create a kotlin file*"	`kotlinc file.kt && kotlin ClassName`
Dart	`.dart`	"*create a dart file*"	`dart file.dart`
R	`.r`	"*create an r file*"	`Rscript file.r`
Shell	`.sh`	"*create a shell file*"	`bash file.sh`
PowerShell	`.ps1`	"*create a powershell file*"	`powershell -File file.ps1`
HTML	`.html`	"*create a html file*"	Opens in browser
CSS	`.css`	"*create a css file*"	—
SQL	`.sql`	"*create a sql file*"	—
Markdown	`.md`	"*create a markdown file*"	—
JSON	`.json`	"*create a json file*"	—

Language-Aware Code Generation

The **Syntax Detector** module ensures the AI generates code matching the language of the open file:

Open File	Say "*create function add*"	Generated Code
`main.py`	→	`def add(a, b): return a + b`
`Main.java`	→	`public int add(int a, int b) { return a + b; }`
`app.js`	→	`function add(a, b) { return a + b; }`
`app.ts`	→	`function add(a: number, b: number): number { return a + b; }`

`main.cpp`	→	`int add(int a, int b) { return a + b; }`
`main.go`	→	`func add(a, b int) int { return a + b }`

📐 How Each Module Works

1. voiceRecorder.ts — Microphone Recording

```
recordAudio(config) → Promise<RecordingResult>
```

- Spawns SoX as a subprocess
- On Windows: uses `-t waveaudio default` to access microphone
- Records at 16kHz, mono, 16-bit WAV (what Whisper expects)
- Returns file path, duration, and file size
- Automatically cleans up temp files after use

2. speechToText.ts — Whisper Integration

```
transcribeAudio(audioFilePath, modelSize) → Promise<TranscriptionResult>
```

- Spawns a Python subprocess
- Loads the specified Whisper model and transcribes audio
- Returns JSON with transcribed text and detected language
- Handles Python/Whisper/FFmpeg errors gracefully
- Runs **locally** — no audio is sent to the cloud (privacy)

3. commandParser.ts — The Brain

```
parseCommand(text) → ParsedCommand
```

- **25+ command types** with priority-ordered regex matching
- Handles natural speech variations and polite prefixes
- Supports spoken numbers (e.g., "*twenty five*" → 25)
- Indian accent phonetic variants (e.g., "*wariable*" → variable)
- First matching pattern wins — pattern order is critical
- Unrecognized commands → `AI_GENERATE` fallback

Pattern Priority Order (prevents ambiguous matches):

```
HELP → UNDO/REDO → DELETE_LINE → STOP_PROGRAM →
REOPEN → OPEN_TERMINAL → SAVE_ALL → FILE_CREATE →
FILE_OPEN → FILE_SAVE → FILE_CLOSE → FILE_RUN →
NAVIGATE → INSERT_CODE → IMPORT → DEBUG/FIX →
RENAME → FOLDER/PROJECT → FILE_DELETE → FIND → COPY/PASTE
```

4. commandDictionary.ts — Help System

```
getAllCommands() → VoiceCommand[]
getFormattedHelp() → string (full reference)
searchCommands(query) → VoiceCommand[]
```

- Stores 120+ commands organized into 10 categories
- Powers the "*help*" and "*show commands*" voice commands
- Each command has phrase, description, aliases, and examples

5. aiEngine.ts — OpenRouter AI

```
generateCode(prompt, config) → Promise<AIResult>
debugCode(code, config, lineNumber?) → Promise<AIResult>
fixError(code, config) → Promise<AIResult>
generateCodeConcept(concept, description, config) → Promise<AIResult>
```

- 3 system prompts optimized for code generation, debugging, and fixing
- Integrates with Syntax Detector for language-aware prompts
- Cleans markdown code fences from AI output
- Comprehensive error handling (401, 402, 429, 503, timeout)

6. syntaxDetector.ts — Language Detection

```
detectLanguage() → LanguageProfile
getProfileByExtension(ext) → LanguageProfile
getAIPromptContext(profile) → string
```

- Detects language from active file extension
- Provides naming conventions, syntax rules, and code templates
- Supports 22+ languages with full profiles (Python, Java, JS, TS, C++, C, Go, Rust, C#, Ruby, PHP, and more)
- Falls back to Python if no file is open

7. vscodeController.ts — The Hands

```
createFile(name, ext)       → Creates file on disk + opens in editor
openFile(name)              → Searches workspace + opens file
saveFile()                  → Saves active editor
closeFile()                 → Closes active tab
runFile()                   → Detects language, runs in terminal
navigateToLine(line)        → Moves cursor to line
insertCode(code)            → Inserts at cursor position
replaceAllCode(code)        → Replaces entire file (for error fixes)
renameSymbol(old, new)      → Find-replace all occurrences
importModule(name)          → Inserts import at top (language-aware)
createFolder(name)          → Creates directory in workspace
deleteFile(name)            → Deletes with confirmation prompt
showOutput(title, content)  → Shows in Output panel
```

8. wakeWordDetector.ts — Wake Word Detection

```
checkWakeWord(text) → 'wake' | 'deactivate' | 'confirm' | 'deny' | 'none'
extractCommandAfterWake(text) → string | null
isOnlyWakeWord(text) → boolean
```

- Uses fuzzy matching and text normalization
- Handles punctuation, capitalization, and accent variations

- 19 wake phrases recognized (including Indian accent variants)
- 19 deactivation phrases recognized
- Hindi confirmation/denial phrases supported

9. voiceFeedback.ts — Text-to-Speech

```
speak(message) → void (queued)
speakAsync(message) → Promise<void>
setEnabled(enabled) → void
setRate(rate) → void
setVolume(volume) → void
```

- Uses OS-native TTS (PowerShell on Windows, `say` on macOS, `espeak` on Linux)
- Message queue prevents overlapping speech
- Pre-built messages for all common actions
- Configurable rate (0-10) and volume (0-100)

10. voiceModeManager.ts — Mode Management

```
startPassiveListening() → starts background wake word detection
activateDirectly() → jumps to ACTIVE mode immediately
deactivate() → returns to PASSIVE mode
stopAll() → stops all listening
```

- Manages state machine: `PASSIVE ⇔ ACTIVE`
- PASSIVE mode: short 2s recordings, checks for wake word only
- ACTIVE mode: longer 7s recordings, full command processing
- Auto-deactivation after 30 seconds of no commands
- Handles safety confirmation flow in ACTIVE mode

11. extension.ts — The Orchestrator

- Registers 7 commands with VS Code
- Creates status bar item (shows recording/mode state)
- Loads environment variables and settings
- Initializes all modules on activation
- Routes parsed commands to the correct controller actions
- Implements the full pipeline: Record → Transcribe → Parse → Execute

---

🧪 Testing Voice Commands

Quick Tests (No Microphone Needed)

Use **Ctrl+Shift+T** (Type Command) for all these tests:

#	Type This	Expected Result
1	`create a python file`	`main.py` created and opened
2	`go to line 5`	Cursor moves to line 5
3	`save file`	Active file saved

4	`insert function to add two numbers`	AI generates and inserts function
5	`run file`	Terminal opens and runs the file
6	`debug error in line 2`	AI analyzes and shows explanation
7	`create folder utils`	`utils/` directory created
8	`import os`	`import os` inserted at top
9	`rename variable x to y`	All `x` renamed to `y`
10	`help`	Shows all available commands

Voice Tests (Microphone Required)

1. Press **Ctrl+Shift+V** (Start Listening)
2. Speak the command clearly
3. Wait for Whisper transcription (2-5 seconds)
4. Click **"Execute"** when prompted
5. Verify the action was performed

Continuous Mode Test

1. Press **Ctrl+Shift+C** (Start Continuous)
2. Say **"Hey Coder"** → status bar should show "*Active*"
3. Say **"create a python file"** → file should be created
4. Say **"save file"** → file should be saved
5. Wait 30 seconds without speaking → should return to "*Passive*"
6. Say **"Hey Coder"** again → should reactivate

⚠️ Troubleshooting

🎤 Microphone Not Detected

Symptoms: "*SoX is NOT installed*" or "*Microphone recording failed*"

Solutions:

1. Verify SoX is installed: `sox --version`
2. Check microphone in Windows Settings → Sound → Input
3. Check Privacy: Settings → Privacy → Microphone → Allow apps
4. Test recording manually:

```
sox -t waveaudio default -r 16000 -c 1 -b 16 test.wav trim 0 3
```

5. Restart VS Code after PATH changes

🗣️ Speech Recognition Failing

Symptoms: "*Could not detect speech*" or Whisper errors

Solutions:

1. Verify Whisper: `python -c "import whisper; print('OK')"`

2. Verify FFmpeg: `ffmpeg -version`
3. **Use a larger Whisper model** — go to Settings → `whisperModel` → `small` or `medium`
4. Speak more clearly and closer to the microphone
5. Increase recording time: Settings → `recordingDurationMs` → `10000`
6. Use **Ctrl+Shift+T** (Type Command) as a fallback

🤖 AI Not Responding

Symptoms: "*OpenRouter API error*" or timeout

Solutions:

1. Check your API key in `.env` — verify it's correct
2. Check credits at https://openrouter.ai/credits
3. Try a different model (e.g., `deepseek/deepseek-chat` or `meta-llama/llama-3.1-8b-instruct`)
4. Check internet connection
5. Error code reference:
   - **401** = Invalid API key
   - **402** = Insufficient credits
   - **429** = Rate limit (wait and retry)
   - **503** = Model temporarily unavailable

❌ Commands Not Executing

Symptoms: Command is recognized but nothing happens

Solutions:

1. Make sure a file is open (required for navigation, save, close)
2. Check the Output panel: View → Output → select "*Voice Coding Assistant*"
3. Check Debug Console (in F5 mode) for error messages
4. Try the same command via **Ctrl+Shift+T** (Type Command) to rule out speech issues
5. Verify the `out/` folder has recent compiled files

🔌 Extension Not Activating

Symptoms: Commands don't appear in Command Palette

Solutions:

1. Make sure `npm run compile` succeeds with zero errors
2. Check that `out/extension.js` exists
3. In F5 mode, check the Debug Console for activation errors
4. Try: `npm install` → `npm run compile` → restart VS Code
5. Check `package.json` for any syntax errors

🔊 TTS Not Speaking

Symptoms: Voice feedback is enabled but no speech

Solutions:

1. Verify the setting is enabled: `voiceFeedbackEnabled` = `true`
2. Check volume: `voiceFeedbackVolume` > 0
3. **Windows:** PowerShell TTS should work out of the box
4. **Linux:** Install espeak: `sudo apt install espeak`

5. **macOS:** The `say` command should work out of the box

License & Credits

- **Whisper** by OpenAI — open-source speech recognition
- **OpenRouter** — unified AI model gateway
- **SoX** — open-source audio processing
- **VS Code Extension API** — by Microsoft

Built with ❤️ for accessibility. Making coding possible for everyone.